Ransomware Mitigation and Network Filtering for IoT using eBPF

CS380S Course Project

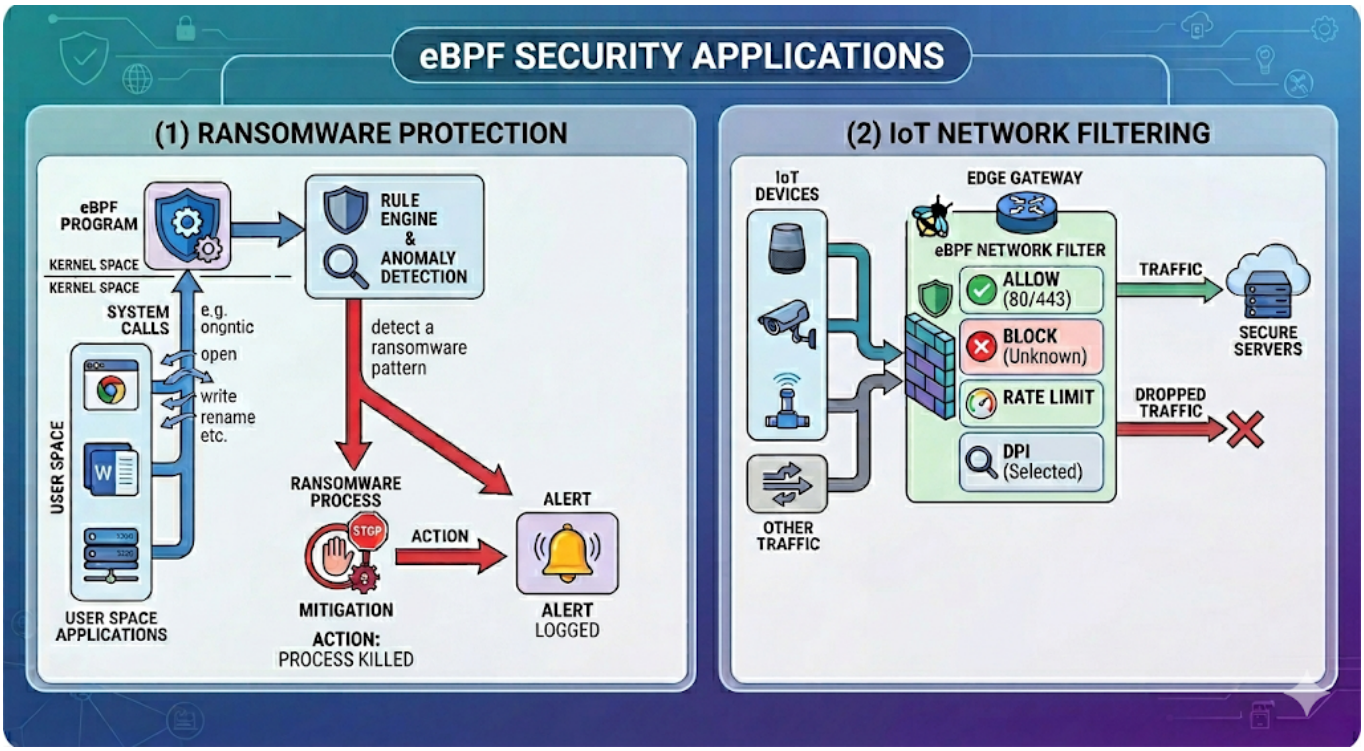
Anish Banerjee ab87743

David Bockelman mb64566

Vaagishwaran Sivakumar vs28699

Danny Tran dlt2836

Jan-April 2026



eBPF Security Applications Explored in our Project (Image by Gemini)

§1. Project Aim

In this project, we aim to study the utility of Extended Berkeley Packet Filter (eBPF) to ensure system security, particularly focusing on two domains:

1. Ransomware detection and mitigation
2. Network packet filtering for Internet of Things

§2. Background

Extended Berkeley Packet Filter (eBPF). eBPF is a revolutionary technology that allows running sandboxed programs in the Linux kernel without changing kernel source code or loading kernel modules. Originally designed for network packet filtering, eBPF has evolved into a general-purpose execution engine for system-wide observability, security, and performance monitoring. By attaching eBPF programs to various hooks such as tracepoints, kprobes, and LSM (Linux Security Modules) hooks, developers can intercept system calls and kernel functions with minimal performance impact.

eBPF v/s BPF. Classic BPF (cBPF) [MJ93] was a minimal in-kernel virtual machine designed solely for packet filtering: it offered a fixed two-register architecture, a limited instruction set restricted to forward jumps, and could only return accept/reject verdicts on network packets.
eBPF, merged into the Linux kernel from version 3.18 onward, is a substantial generalization. It extends the register file to ten 64-bit general-purpose registers, supports arbitrary control flow including bounded loops (verified at load time), and provides a rich helper-function API for interacting with kernel data structures. Crucially, eBPF programs can attach to a wide variety of hooks beyond networking—tracepoints, kprobes, uprobes, perf events, LSM hooks, and cgroup callbacks—making it a general-purpose kernel extension mechanism [CTJS20]. The eBPF verifier statically checks every program before loading to guarantee memory safety, bounded execution, and no kernel crashes, which is what makes it safe to run user-supplied code in kernel context. For our project, these capabilities are essential: cBPF could filter packets but could never intercept `write` syscalls, sample data buffers, or maintain per-process state in hash maps—all of which our ransomware monitor relies on.

Ransomware. Ransomware is a class of malware that disrupts system operations by encrypting a victim's files, effectively holding their data hostage until a ransom is paid. The attack life-cycle typically involves initial infection, lateral movement, and finally, the encryption phase where user-generated content (documents, photos, databases) is encrypted using strong cryptographic algorithms like AES or RSA.
Detection of ransomware is inherently difficult because the act of reading and writing files is a standard behavior for many legitimate applications. However, ransomware exhibits distinct "signals" during its execution that can be monitored to catch them:

- *High Entropy Writes*: Encrypted data lacks the structure of plain text or standard file formats, resulting in high Shannon entropy (approaching 8 bits per byte).
- *High-Frequency I/O*: To maximize the amount of data encrypted before being caught, ransomware often processes files at a much higher rate than a human user or typical background task.
- *Massive File Deletions/Renames*: Some ransomware strains delete the original file after creating an encrypted copy or move files across directories.
- *File Extension Changes*: Many variants rename files to include specific extensions (e.g., `.locked`, `.crypto`) or create temporary files with these extensions, to signal that the data has been encrypted.

Distributed Denial of Service (DDoS). DoS attacks overwhelm a target system — such as a server, website, or network — with traffic, preventing legitimate users from accessing it. Distributed DoS (DDoS) attacks extend this by coordinating traffic from many distributed machines simultaneously, making them significantly harder to detect, filter, and mitigate.

Internet of Things (IoT). IoT refers to physical devices equipped with sensors, computing capabilities, and software that connect to networks and communicate data over the Internet. With billions of deployed devices typically designed to be resource-constrained and lacking robust security, IoT represents a large and vulnerable attack surface. Thus are susceptible to DDoS attacks and prime targets for botnet recruitment for DDoS attacks — as demonstrated by the Mirai botnet [AAB+17].

§3. Prior work

Ransomware Detection and Mitigation. Ransomware actors routinely exploit high-severity CVEs to compromise enterprise infrastructure. Notable examples include CVE-2021-44228 (Log4Shell), which enabled unauthenticated remote code execution and was leveraged by LockBit [CIS23b]; and CVE-2026-20131 (CVSS 10.0), an insecure deserialization flaw in Cisco Firewall Management Center exploited as a zero-day by the Interlock ransomware group for over a month before disclosure [Cis26]. the ProxyShell chain (CVE-2021-34473, CVE-2021-34523, CVE-2021-31207), which allowed pre-authenticated RCE on Microsoft Exchange [CIS22]; and CVE-2023-28252, a Windows CLFS driver privilege escalation exploited by Nokoyawa ransomware [CIS23a].
Conventional defenses against such threats fall into several categories, each with well-documented limitations:

- *Signature-based detection* matches files against a database of known malware patterns, but fails against novel or polymorphic variants [IIBI24].

- *Static analysis* examines binaries without execution, but is defeated by obfuscation and code mutation [AA23].
- *Dynamic sandbox analysis* observes program behavior in an isolated environment, but is slow and vulnerable to sandbox-evasion techniques [AA23].

To address these gaps, researchers have turned to eBPF, which enables low-overhead, kernel-level behavioral monitoring without kernel modification. [ZTP+23] demonstrated eBPF's viability for tracking ransomware-relevant syscalls and network activity. [HK23, ZD23] built a real-time eBPF-based detection systems that collect file operation events and apply ML classification. [PVMJ24] further proposed running decision tree and MLP inference directly inside the kernel, showing that in-kernel detection is critical given the speed of modern encryption. [SKK25] combined eBPF tracepoints with NLP-based ransom note analysis, achieving 99.76% detection accuracy within seconds.

Network Filtering in Resource-Constrained Settings. Traditional DDoS mitigation techniques, including access control lists (ACLs), rate limiting, and signature-based intrusion detection and prevention systems (IDS/IPS) [R+99, BBCL19], have been widely adopted in conventional network infrastructures. However, these approaches often rely on substantial compute and memory resources, limiting their applicability in resource-constrained environments such as IoT devices. More recently, deep learning-based methods for detecting anomalous traffic patterns [NSJ16, AS25] have demonstrated strong performance, but their high computational overhead makes them impractical for deployment at the network edge. To address these limitations, prior work has explored lightweight, in-kernel programmability through technologies such as eBPF, which enables safe and efficient packet processing within the Linux kernel [VCP+20]. When combined with XDP, which operates at the earliest stage of packet reception, these systems can filter malicious traffic before it traverses the network stack, enabling high-performance firewalling and intrusion detection with minimal overhead [HAC+24]. We follow these methods and use eBPF with XDP to filter packets [Tol25].

Adaptive Blocking and Rate Limiting. Many network filtering systems rely on blocking source IP addresses that exceed predefined thresholds. However, static or permanent IP blocking has been shown to be ineffective in dynamic network environments. Legitimate users may be inadvertently penalized due to shared IP addresses (e.g., NAT) or frequent IP reassignment, while adversaries can evade blocking by rotating source addresses [RHM+20, FBD+11]. To mitigate these issues, prior work has proposed adaptive blocking strategies, including time-to-live (TTL)-based bans and sliding window rate limiting, where offending IPs are blocked for a limited duration that may be extended upon repeated violations [Ell14]. These approaches reduce collateral damage to other users while still implementing security. We take inspirations from these methods to implement temporary ban on IPs that exceed a certain threshold for sent packets.

Spoofing and Reflection Attacks. A significant challenge for IP-based filtering arises from IP spoofing and reflection attacks, which are especially prevalent in UDP-based protocols [FS98, LXLG15]. In such attacks, adversaries forge source IP addresses to obscure their origin and exploit third-party services, such as DNS or NTP servers, to amplify traffic directed at a victim. As a result, filtering decisions based solely on source IP can be unreliable, as the apparent sender may not correspond to the true origin of the traffic. Prior work has explored additional heuristics and signals, including request-response consistency checks [HHF26], traffic asymmetry detection [HHF26, ZLW+20], and protocol-specific anomalies [KMGG07], to distinguish malicious traffic from legitimate requests.

Lightweight Connection Validation. For connectionless protocols such as UDP, the absence of inherent session state further complicates traffic filtering and attack mitigation. To address this, prior work has introduced lightweight connection tracking mechanisms that maintain minimal per-flow state, such as packet counts or recent timestamps, within kernel-level data structures [Wel04, FMMR10]. We take inspiration from these works and track tuples of requests with src/dst IP, src/dst port, DNS id. In addition, several systems employ token-based or stateless validation schemes, where clients must correctly respond to server-issued cryptographic tokens (e.g., SYN cookies) before being allowed to access resource-intensive services [PDF20, YCR24]. These mechanisms validate that a client can receive responses at its claimed source address, thereby mitigating spoofing-based attacks. Do you

§4. Our Implementation

4.1. Ransomware Detection and Mitigation

Our approach to ransomware detection and mitigation utilizes a dual-component architecture consisting of a kernel-space eBPF monitor and a user-space analysis engine.

Feature	REvil	HelloKitty	WannaCry
Unlink	High	Med	High
Urandom	High	Low	Low
Pkill	High	High	High
Rename	High	High	High
Write	High	High	High
OpenSSL	Low	High	High
CPU utilization	High	High	High
Openat	High	High	High

Table 1: Primary Syscall Invocations Across Ransomware Variants During Ransomware Execution [SKK25]

4.1.1. Kernel-level Instrumentation (`monitor.bpf.c`).

We use eBPF kprobes to intercept critical kernel functions related to file I/O and process management, according to Table 1. Specifically, we hook into:

1. `do_sys_openat2`: To monitor file opens, track filenames for write correlation, and detect access to `/dev/urandom` or `/dev/random` (emitted as a distinct `URANDOM_READ` event type, since ransomware reads from the kernel entropy pool to generate encryption keys).
2. `ksys_write`: To capture data buffer samples and track the volume of data being written.
3. `do_renameat2`: To detect mass renaming of files from their original names to custom encrypted file extensions.
4. `do_unlinkat`: To monitor high-frequency file deletions, a common post-encryption behavior. The ransomware variants often create temporary `.lck` or `.temp` files as intermediary files between the original file and the final encrypted file. After creating the final encrypted file, the ransomware deletes these temporary files using the unlink or unlinkat system calls.
5. `iterate_dir`: To detect rapid directory traversal, a signature of ransomware scanning for targets before encryption.
6. `do_send_sig_info`: To intercept `SIGKILL` and `SIGTERM` delivery, capturing the target process name. Ransomware typically terminates all running processes before encrypting them, and kills open file handles, ensuring no interruptions occur during the file encryption process.

The eBPF program is written in restricted C and utilizes `BPF_PERF_OUTPUT` to stream event data to user-space. For each intercepted call, we collect the Process ID, parent PID, the process name, the target filename, and a 128-byte sample of the write buffer. To stay within the kernel's stack limit, we use a `BPF_PERCPU_ARRAY` as a scratch buffer for event data.

4.1.2. User-Space Detector (`detector.py`).

A Python-based agent, built using the BCC library, processes the stream of eBPF events and applies several detection heuristics:

1. *Entropy Analysis*: For every write event, we calculate the Shannon entropy of the sampled buffer. A threshold of $\sim$ 6 bits/byte was chosen to lie above typical plaintext (3.5–5.0 bits/byte) and structured binary data, while remaining distinguishable from legitimately compressed data ($\sim$7.5–7.8 bits/byte). If a process consistently writes data exceeding this threshold across multiple files, it is flagged as potentially performing encryption. *This is a characteristic invariant of ransomware, and all other heuristics are gated by this check.*
2. *Frequency Tracking*: We maintain a sliding time window ($\sim$ 1 second) for each process. If a process exceeds a threshold of writes per window, those writes target multiple distinct files, and the average entropy is high, the process is classified as suspicious.
3. *Extension Filtering*: The engine maintains a list of suspicious extensions (e.g., `.locked`, `.crypto`). Any `openat` or `rename` event involving these extensions triggers an immediate alert.
4. *Directory Traversal Detection.* Rapid directory traversal combined with concurrent write activity arm a short-lived "scan-then-modify" context that is only promoted when stronger evidence later appears, such as high-entropy file writes or in-place overwrite behavior. Directory scanning alone (e.g., `find`) still does not trigger, and this change substantially reduces false positives from benign recursive workflows.
5. *Canaries.* Hidden canary files are deployed in sensitive directories. Any non-whitelisted process accessing a canary triggers an immediate critical alert regardless of other heuristics, providing a low-false-positive tripwire.

False-Positive Reduction. A key challenge in behavioral ransomware detection is that many legitimate applications—compilers, compression tools, video encoders, encryption utilities, databases—produce I/O patterns that overlap with ransomware signals. We addressed this through a layered set of techniques that shift detection from simple signal thresholds toward *behavioral pattern matching*:

1. *Process Whitelist with Integrity Verification.* A configurable set of trusted process names (e.g., `git`, `apt`, `gzip`, `zstd`, `gpg`, `ccencrypt`, `openssl`, `rsync`, `logrotate`, and `postgres`) is skipped during analysis. To prevent adversaries from abusing whitelisted names (e.g., by naming a malicious binary `gzip`), the whitelist is hardened with two checks:

   (a) *binary hash verification*, which computes the SHA-256 of `/proc/<pid>/exe` and compares it against known-good digests, and

   (b) *process lineage validation*, which walks the parent-process chain via `/proc/<pid>/status` and requires at least one trusted ancestor (e.g., `bash`, `systemd`, `make`). Either check failing revokes whitelist membership.

   The trust policy explicitly favors suppressing false positives from single-file invocations of common encryption and archival tools over attempting to classify those invocations as ransomware on behavior alone. In the checked-in detector, the built-in whitelist contains 32 entries for broader false-positive risk (see Appendix B). However, the measured test context

admits a smaller five-entry policy, `gpg`, `gzip`, `zstd`, `ccencrypt`, and `openssl`, that was sufficient to preserve the experiment outcomes (see Appendix C).

2. *Write Target Classification.* Writes to system paths (`/dev/`, `/proc/`, `/sys/`, etc.) are excluded from all heuristics. This prevents false positives from disk defragmenters writing to block devices or databases writing to `/var/lib/`. Though it may appear to be insecure against ransomware, in practice this is a lower risk than it sounds, for two reasons:

   (a) System paths are already protected by other mechanisms. Writes to `/dev/sda` require root. Writes to `/proc/` and `/sys/` are virtual filesystems that don't store user data. `/var/lib/` is protected by file permissions. A process that has root access to write to block devices has already compromised the system beyond what a user-space monitor can defend against.

   (b) Ransomware targets user data, not system files. The business model depends on encrypting files the victim can't recreate — documents, photos, databases. System binaries and configs can be reinstalled. Real-world ransomware families (LockBit, Conti, BlackCat) overwhelmingly target `/home`, `/srv`, mounted shares, and database data directories, not `/dev/` or `/proc/`.

3. *File Diversity Scoring.* Rather than alerting on raw write frequency alone, we track how many *unique file paths* and *unique parent directories* a process writes to within the time window. A defragmenter writing to the same file 50 times produces no diversity alert; ransomware encrypting files across `~/Documents`, `~/Pictures`, and `~/Desktop` does. The use of both path-level and directory-level diversity is deliberate: path diversity alone could be gamed by ransomware that encrypts many files within a single directory.

4. *In-Place Overwrites and Magic-Byte Alerts.* We detect when a process writes high-entropy data back to a file it previously opened (in-place overwrite) and when write buffers lack recognized file-type magic bytes. The eBPF layer emits open events for both creations and plain opens, allowing these checks to observe true in-place modification paths rather than depending only on `O_CREAT`. Both checks require the process to have overwritten *at least two distinct files* before alerting. This threshold is the critical design choice that separates legitimate single-file encryption (`ccencrypt` on one file) from ransomware behavior (encrypting many files in-place).

5. *Output-to-Input Correlation.* Legitimate tools produce output with predictable naming conventions, e.g. `data.csv` → `data.zip`. Ransomware renames files to unrelated extensions (`.locked`, `.a1b2c3`). The detector maintains a list of known legitimate output suffixes and suppresses in-place overwrite alerts when the output name is a plausible derivative of a recently-opened input file.

6. *Write-Then-Delete Correlation.* We track recent high-entropy write targets per process. When an `unlink` deletes a file that is *not* among the recent write targets (i.e., the original plaintext file rather than the encrypted output), and the process has written to three or more distinct files recently, a critical alert fires. The threshold of three is calibrated to ensure single-file compression workflows (write `.gz`, delete source) pass silently, while bulk encryption-then-deletion is flagged.

Fine-Grained Tightening. We implemented a few more advanced heuristics to capture specific ransomware attackers.

1. *Process-Tree Attribution of Delegated Writes.* We extended the detector to cover a specific blind spot: a non-whitelisted ransomware orchestrator delegating encryption to helper child processes. The eBPF monitor now captures each event's parent PID in-kernel at syscall time. When a child performs a high-entropy write, the detector can attribute that write to the nearest *eligible* non-whitelisted ancestor - whose recent scan/write context overlaps the child's target paths - and then re-evaluate the parent's heuristics. This attribution is not tied to the child's whitelist status; instead, the design treats the eligible parent as the meaningful subject while still avoiding loose inheritance through generic launchers such as `bash` and `python3`.

2. *Slow-Burn Detection via Cumulative Profiling.* All of the heuristics above operate within a sliding time window (default 1 s). A ransomware variant that encrypts one file every few minutes—a "slow-burn" attack—would never accumulate enough events in any single window to trigger an alert. To address this, we maintain a *cumulative per-process profile* that is never pruned while the process lives. Each high-entropy write to a new meaningful user file adds $+1$ to the profile score; each new parent directory adds $+2$; each deletion of a meaningful user file the process did not write adds $+3$; each `/dev/urandom` read adds $+3$ (first access only); each `SIGKILL`/`SIGTERM` sent to another process adds $+4$; and each in-place overwrite adds $+5$. When the score crosses a configurable threshold (default 15), a critical "Slow-burn ransomware" alert fires *only if* the process has an *entropy-backed anchor*, such as repeated high-entropy writes or an in-place overwrite. Duplicate writes to the same file are counted only once, temp-like paths and low-entropy writes are ignored, and whitelisted processes never accumulate a profile.

   Additionally, a `/dev/urandom` read triggers an *immediate* alert if the same process has recently performed two or more high-entropy writes to distinct user files within the sliding window—the "generate-key-then-encrypt" pattern. Kill signals are still tracked in the cumulative profile, but no longer trigger ransomware alerts by themselves.

4.1.3. Mitigation (`mitigator.py`).

The mitigation subsystem supports three configurable action modes: `simulate` (log only, for tuning), `suspend` (`SIGSTOP` to freeze the process for administrator review), and `kill` (`SIGKILL` for immediate termination). The `suspend` mode is recommended for production, as it halts damage while preserving forensic state.
Modern Endpoint Detection and Response (EDR) platforms such as CrowdStrike Falcon and SentinelOne employ a graduated response chain that goes well beyond process termination [Cyb24]. Inspired by these production tools, our mitigator implements a 5-step response chain that executes sequentially when the action mode is `suspend` or `kill`:

1. *Process Kill.* The process tree is terminated by first signaling all child PIDs (discovered via `/proc/<pid>/task/<pid>/children`) and then the main PID. In `suspend` mode, `SIGSTOP` is used instead of `SIGKILL`, freezing the process for administrator review while preserving its memory state for forensic analysis.

2. *Quarantine.* Since step 1 has already killed the process, the binary is no longer memory-mapped and can be safely *moved* from its original location to a quarantine directory (`/var/lib/ransomware-monitor/quarantine/` by default). All permission bits are stripped from the quarantined copy (`chmod 000`), and the original path is added to a persistent blocklist.

3. *Network Isolation.* The process's real UID is read from `/proc/<pid>/status`, and an iptables `OUTPUT` rule is installed using the `owner` match module to drop all outbound traffic from that UID. This prevents data exfiltration and C2 communication from any other processes running under the same user, which is important because ransomware often exfiltrates data before encrypting it as part of a double-extortion strategy. This step is opt-in via `--enable-network-isolation`.

4. *Remediation.* All files the flagged process modified (tracked via per-PID write logging in the detector) are enumerated and logged. In the current implementation this produces a manifest for manual or automated recovery; a full implementation would restore files from backup or undo renames.

5. *Rollback.* On the first critical-severity alert in a session, a configurable shell command is executed to trigger a filesystem snapshot. This integrates with Copy-on-Write filesystems such as Btrfs (`btrfs subvolume snapshot`) or ZFS (`zfs snapshot`), providing a point-in-time recovery image with near-zero data loss. The snapshot fires only once per session to avoid flooding the filesystem during a sustained attack.

The response mode is selectable via `--action-mode`: `simulate` logs all steps without executing them (suitable for tuning and dry runs), `suspend` uses `SIGSTOP` at step 1 for human-in-the-loop review, and `kill` executes the full chain with `SIGKILL`.

4.2. Network Filtering for IoT Edge Devices

eBPF/XDP-Based Packet Filtering. An eBPF program attached in XDP native (driver) mode performs early, stateful packet filtering directly within the network driver, before `sk_buff` allocation. For each incoming packet, the source IP is first checked against a `block_map` that stores temporarily banned addresses along with their expiration timestamps; packets from currently blocked sources are immediately dropped via `XDP_DROP`. For non-blocked sources, the program maintains a per-IP packet counter in a `rate_map` using a sliding time window. If the packet count exceeds a configurable threshold within this window, a temporary block entry is installed in the `block_map`, and subsequent packets from that source are dropped at the XDP layer. Otherwise, packets are allowed to pass `XDP_PASS`. To avoid permanent bans and reduce false positives, block entries are automatically expired after a fixed time-to-live (TTL), with a periodic sweeper removing stale entries. By operating in XDP native mode, this approach enables high-throughput, low-latency mitigation by dropping malicious traffic at the earliest point in the packet processing pipeline without requiring user-space intervention.

XDP-Based Connection Tracking for Spoofing Mitigation. While threshold-based rate limiting is effective against high-volume sources, it is insufficient in adversarial settings where attackers distribute traffic across many source IPs or spoof IP addresses to evade per-IP limits. In such cases, each individual source may remain below the detection threshold, allowing the attack to bypass purely rate-based defenses. This is particularly relevant for reflection and amplification attacks (e.g., DNS), where attackers can forge source IPs and generate large volumes of unsolicited responses toward a victim.
To address this limitation, we implement lightweight connection tracking directly in XDP to validate whether incoming responses correspond to legitimate outgoing requests. Specifically, for each outgoing request (e.g., a DNS query), the XDP program extracts and records a tuple consisting of the source and destination IPs, source and destination ports, and a protocol-specific identifier (such as the DNS transaction ID). This tuple is stored in an eBPF map representing pending requests. When a response packet arrives, the program parses its headers and checks whether a matching tuple exists in the map. If a match is found, the response is considered valid and is allowed to pass; otherwise, it is treated as unsolicited and dropped at the XDP layer.
This design effectively filters spoofed or reflection-based traffic by enforcing a request-response consistency check, rather than relying solely on source IP reputation or rate thresholds. Additionally, entries in the pending-request map are maintained with a short expiration time to bound

memory usage and ensure stale requests do not permit delayed or replayed responses. By performing this validation in XDP native mode, the system rejects invalid traffic at the earliest point in the packet processing pipeline, enabling robust defense against distributed and spoofed attacks with minimal overhead.

Mitigating TCP SYN Flood Attacks with XDP SYN Cookies. While connection tracking is effective for validating request-response consistency in protocols such as DNS, it is not directly applicable to services that must accept unsolicited incoming connections, such as web servers. In these settings, TCP SYN flood attacks pose a significant threat. An attacker can send a large volume of SYN packets, often with spoofed source IP addresses, causing the server to allocate state for half-open connections. This exhausts memory and connection tables, preventing legitimate clients from establishing connections. Since these attacks may involve many distributed or spoofed sources, simple per-IP rate limiting is insufficient.
To address this, we implement SYN cookie-based validation directly at the XDP layer, enabling stateless handling of TCP connection establishment. Upon receiving a SYN packet, instead of allocating connection state in the kernel (e.g., `sk_buff` structures or socket state), the XDP program constructs a SYN-ACK response in-place. The initial sequence number of this SYN-ACK is derived from a cryptographic hash over the connection tuple (source/destination IP and ports), a local secret, and a timestamp. This response is transmitted immediately using `XDP_TX`, avoiding any per-connection memory allocation.
When the client responds with an ACK, the XDP program recomputes the expected hash and verifies that the acknowledgment number matches the encoded sequence number (i.e., `ack = hash + 1`). Only if this validation succeeds is the packet allowed to proceed up the network stack (`XDP_PASS`), at which point the kernel can safely allocate connection state for a legitimate client. Otherwise, the packet is dropped.
By deferring state allocation until after handshake validation, this approach eliminates the memory exhaustion vector exploited by SYN flood attacks. Furthermore, because the validation is performed entirely in XDP native mode, malicious SYN traffic is handled at the earliest point in the packet processing pipeline, providing strong protection against high-rate, distributed, and spoofed attacks with minimal overhead.

§5. Experiments and Evaluation

5.1. Ransomware Setup and Results

Experimental Setup. We evaluate the ransomware monitor using a small experiment harness that runs labeled scenarios in isolated per-run sandbox directories. Each scenario is specified as a row in a CSV file with a label, an expected process name, and a workload command. For each run, the harness launches the monitor, executes the workload inside the sandbox, collects structured alert output from the detector, and writes a results file containing the scenario label, whether the monitor predicted positive or negative, and the supporting logs needed for later analysis. This gives us a uniform pipeline for comparing very different workloads while still computing standard metrics such as true positives, false positives, true negatives, and false negatives.

Four Complementary Test Families. Our ransomware evaluation is intentionally split across four complementary families of tests, each probing a different aspect of the detector:

- *Legacy Suite.* Self-written control scenarios that disentangle individual heuristics such as suspicious extensions, entropy, rename behavior, and unlink correlation.
- *Atomic Red Team T1486 Suite.* Uses the official Linux *Atomic Red Team* tests for "Data Encrypted for Impact," giving small, tool-level ATT&CK-aligned adversary emulations [Red26].
- *Benign Stress Suite.* Derived from Phoronix-style workloads, using legitimate compression, encryption, media, and compilation tasks to measure false positives under realistic high-I/O activity.
- *Behavioral Suite.* Contains ransomware-inspired process behaviors drawn from well-studied Linux ransomware families, testing whether a non-whitelisted process that behaves like ransomware is detected even when simple tool identity is no longer informative.

Together, these four families let us separate heuristic correctness, tool-level adversary emulation, benign workload robustness, and full behavioral detection. For details regarding these test suites, see Appendix A.

Results. Tables Table 2 and Table 3 summarize the full tuned run of the ransomware monitor. These results use the final tuned detector configuration evaluated across all four ransomware suites: legacy controls, Atomic Red Team T1486, benign stress workloads, and behavioral ransomware simulations.

Discussion. The tuned results now show a detector operating at a much cleaner policy boundary. The overall specificity is 1.000: the benign stress suite is fully quiet, and no false positives remain in the combined run. The cost is explicit in recall. Under the current trust policy, the Atomic Red Team T1486 suite becomes an all-miss boundary because those scenarios are single trusted-tool invocations on one file rather than multi-file ransomware workflows. The quality of the behavioral detection remains the strongest result: the behavioral suite contributes 24/24 true positives across both the original ransomware-inspired workloads and the higher-fidelity in-place overwrite variants. This means the detector continues to fire

Run	TP	FP	FN	TN	Precision	Recall	Specificity	Bal. Acc.
Full tuned combined run	27	0	18	36	1.000	0.600	1.000	0.800

Table 2: Overall ransomware-monitor performance across all four ransomware suites.

Suite	Runs	TP	FP	FN	TN
Legacy	21	3	0	6	12
Atomic T1486	12	0	0	12	0
Benign Stress	24	0	0	0	24
Behavioral	24	24	0	0	0

Table 3: Suite-level breakdown for ransomware evaluation.

when the workload removes cosmetic signals such as suspicious extensions, renamed outputs, and ransom-note-like artifacts, as long as the process still exhibits the invariant file-system behavior of ransomware.

Extended Live Monitoring. To validate the detector beyond scripted workloads, we ran the monitor continuously on a development workstation (WSL2, Ubuntu 20.04) during a normal working session spanning several hours. During ordinary usage, the monitor produced zero false positives. However, running `sudo apt update` triggered a burst of alerts from three sources: (1) the `http` transport worker (a child of `apt` located at `/usr/lib/apt/methods/http`) downloading `.xz`-compressed package lists at high frequency with entropy $\sim$6.55, (2) the `apt-check` helper performing rapid file deletions during cleanup, and (3) `dpkg` having its whitelist revoked because its process lineage passed through apt helper processes (`cnf-update-db`, `apt-key`) that were not in the trusted-parent set.
These findings motivated us to expand the whitelist and trusted-parent set to cover apt's transport workers (`http`, `https`, `gpgv`), helpers (`apt-check`, `apt-config`, `apt-esm-json-hook`), and parent processes (`apt-key`, `cnf-update-db`) so that the lineage verification correctly identifies these as legitimate.
However, high throughput installers may still trigger false positives for our monitor, since they too write high entropy content at high frequency. Despite several suppression attempts, reliably distinguishing *all* benign installer activity from ransomware behavior remains an open challenge that we leave for future work.

5.2. Network Filtering Setup and Results

Experimental Setup. Our first setup emulates a distributed attack scenario in a controlled setting. Conceptually, the threat model consists of an attacker generating high-rate traffic from multiple or spoofed sources to overwhelm a resource-constrained IoT device. In practice, we use a Raspberry Pi as the IoT device victim and a single attacker machine connected via direct Gigabit Ethernet. We generate UDP flood traffic via iperf3 at up to 100 Mbps (attacker). The attacker generates both benign and adversarial traffic, emulating multiple sources through parallel flows and configurable patterns. We set the rate limit at 800 packets per second.
Our second setup aims to emulate a reflection attack. Conceptually, an attacker sends spoofed DNS requests to a DNS server, causing the server to reflect amplified responses toward a victim device. To emulate the reflection attack scenario, we construct a local testbed in which a client machine generates both benign DNS queries and a high volume of malicious, spoofed DNS requests. These requests are sent through an intermediary host that forwards them to Google DNS, thereby reproducing the reflection behavior observed in real-world attacks. Critically, the spoofed packets are crafted such that the source IP address corresponds to the target device (the Raspberry Pi). As a result, DNS responses are reflected back to the target, causing it to receive a mixture of legitimate responses and amplified malicious traffic.
Our third setup evaluates our TCP SYN flood mitigation. The attack consists of having our attacker (the laptop) generate 200Mbps of TCP SYN packets directed at our target (the Raspberry Pi) using `hping3`. We also have the Pi host a `netperf` server, which the laptop then uses with the `netperf` client in `tcp_crr` mode to test how many TCP 3-way handshakes it can complete with the Pi over a period of 20 seconds. We then plot the tran (connections) per second under the 4 different combinations of the XDP SYN cookies and the SYN flood attack being on or off. We also plot the CPU utilization using `mpstat` during the attack with the SYN cookies defense on and off.
We evaluate mitigation rate, CPU utilization, and system stability under attack.

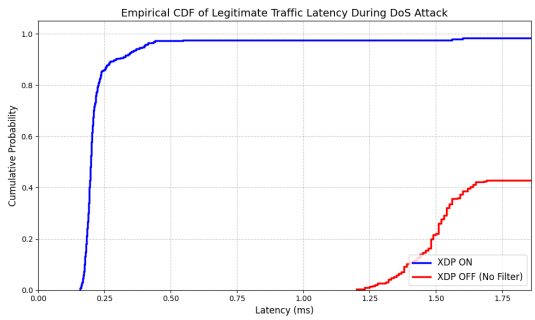


Figure 1: **Empirical CDF of latency under DoS attack.** XDP filtering (blue) preserves low latency for legitimate traffic, while disabling filtering (red) leads to significant latency increases due to resource contention.

Results. In this first experiment, we measure the latency of legitimate requests under a sustained DoS attack by

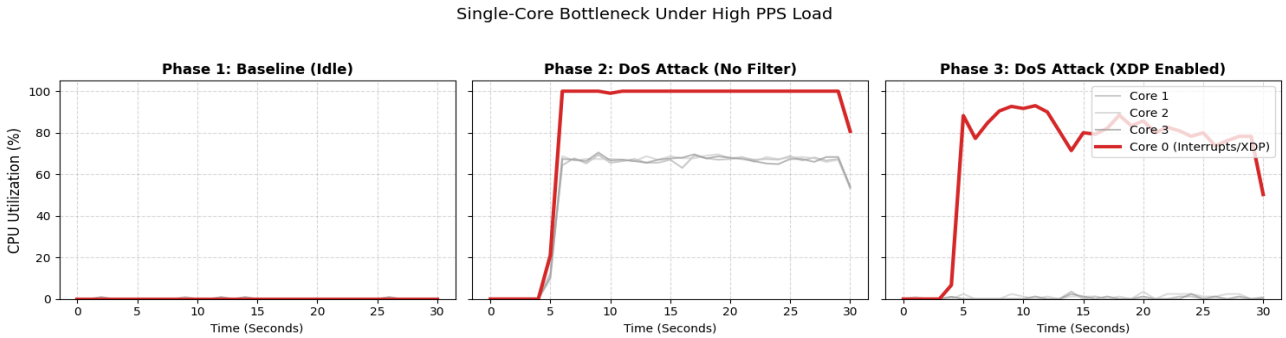


Figure 2: **CPU utilization under high packet-rate DoS load.** Without filtering, the DoS attack saturates CPU resources across cores. With XDP enabled, packet processing is concentrated at the early interrupt/XDP path, preventing the attack from consuming additional application-level resources.

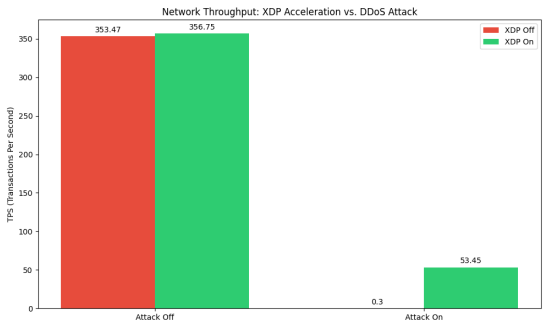


Figure 3: **TCP transactions per second under different setups.** The TCP SYN cookie defense does not have a significant effect on TPS while not under a TCP SYN flood, indicating minimal overhead in typical operation. While under a TCP SYN flood attack, the TPS for both the TCP SYN cookie defense on and off decrease sharply, but the TCP SYN cookie defense provides significant throughput gains as opposed to the base configuration.

comparing a baseline system without filtering to one with XDP-based packet filtering enabled. A high-rate flood of malicious traffic is generated concurrently with benign requests to evaluate how each system handles resource contention. As shown in Figure Fig. 1, enabling XDP filtering preserves low latency for legitimate traffic, as malicious packets are dropped at the earliest point in the network stack. In contrast, without filtering, attack traffic is allowed to propagate through the system, consuming CPU and memory resources and causing significant queuing delays. This results in a substantial shift toward higher latencies for benign requests, highlighting the importance of early-stage mitigation in maintaining quality of service under adversarial load.

Figure Fig. 2 illustrates CPU utilization across cores under three phases: idle, DoS attack without filtering, and DoS attack with XDP enabled. In the baseline phase, CPU usage remains negligible across all cores. During the DoS attack without filtering, one core becomes fully saturated while the remaining cores also exhibit high utilization, indicating that malicious traffic propagates through the network stack and consumes system-wide resources. In contrast, when XDP filtering is enabled, CPU usage is primarily concentrated on the core handling early packet processing (interrupt/XDP path), while the other cores remain largely idle. This demonstrates that XDP effectively contains the attack at the earliest stage of packet processing, preventing it from spreading across the system and reducing overall resource contention. As a result, application-level processing remains unaffected, enabling the system to maintain responsiveness under high packet-rate attacks.

For the second experiment, we evaluate system performance under a DNS reflection attack, where a high volume of amplified UDP responses is directed toward the server, simulating a realistic spoofing-based attack scenario. Legitimate DNS queries are issued concurrently to measure the impact on service latency. Figure Fig. 5 shows that enabling XDP filtering significantly improves performance, with the majority of queries completing at low latency. In contrast, without filtering, the amplified traffic overwhelms the system, leading to a substantial increase in latency and a broader distribution of response times. These results demonstrate that early packet filtering is critical in mitigating reflection-based attacks, as it prevents amplified traffic from consuming network and compute resources, thereby preserving service quality for legitimate clients. Figure Fig. 6 shows that XDP filtering prevents CPU saturation by dropping amplified traffic early, reducing the processing burden during the reflection attack.

Finally, in the third experiment, we see mixed results in terms of the overall effectiveness of the TCP SYN cookie defense in Fig. 3. When we analyze the throughput of the system among the 4 different regimes in terms of TPS, we see that the TCP SYN cookie defense has nearly zero overhead in normal operation, and provides significant throughput improvements over the base system while under a TCP SYN flood attack. However, comparing the attack TPS to the normal operation TPS shows that throughput is still significantly affected by the TCP SYN flood attack, which would still exhibit DDoS-like symptoms to the user. Additionally, in Fig. 4, we see that while overall CPU utilization decreases with the TCP SYN flood

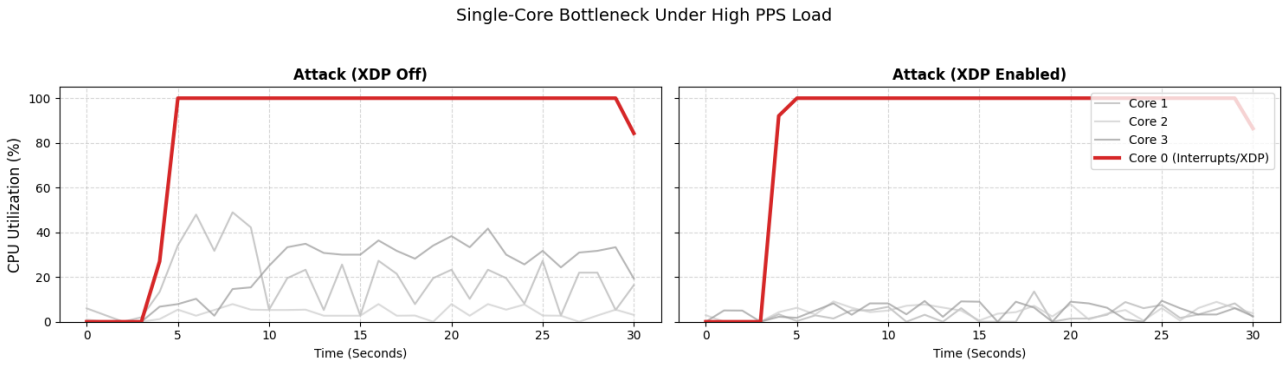


Figure 4: **CPU utilization under TCP SYN flood attack.** In both scenarios, the interrupt core (core 0) is completely saturated by the attack. However, we can see minimal gains in the other cores with the TCP SYN cookie defense enabled.

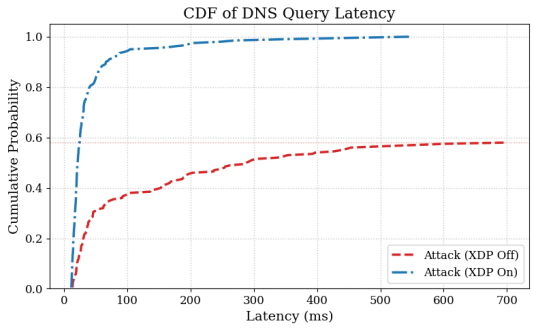


Figure 5: **CDF of DNS query latency under reflection attack.** With XDP enabled (blue), most queries complete with low latency, while disabling filtering (red) results in significantly higher latency due to amplified traffic overwhelming the system.

defense, the attack is still able to completely saturate the main interrupt core (core 0), which, in machines like the Pi with only a single RX queue, would again exhibit DDoS behavior to the user. We find that the results prove the concept of the TCP SYN cookie defense, they fail to show the practicality of the defense on resource constrained edge devices like the Pi.

§6. Limitations and Future Work

Ransomware Detection. Our behavioral heuristics are effective against ransomware that exhibits bulk encryption patterns (many files, multiple directories, high entropy) but cannot detect *single-file* encryption operations that are behaviorally identical to legitimate tool usage. The Atomic Red Team T1486 tests illustrate this: each test encrypts exactly one file using a standard tool (`gpg`, `7z`, etc.), producing I/O indistinguishable from benign usage. Slow-burn attacks that spread encryption over minutes or hours are now addressed by the cumulative per-process profile, though an attacker that restarts under a new PID for each file could evade per-PID tracking. The process whitelist, while hardened with binary hash and lineage checks, relies on `/proc` availability and cannot protect against attacks that compromise the kernel itself.
Directions for future work include:

- *Performance overhead characterization.* Measuring the CPU, memory, and I/O latency overhead of the monitor under sustained workloads (e.g., large compilations, database transactions, media encoding) would quantify the cost of always-on protection and identify whether the current kprobe-based instrumentation introduces measurable slowdowns on I/O-heavy systems.

- *Cross-PID correlation.* An attacker that forks a new process for each file encryption would evade per-PID cumulative profiling. Correlating behavior across process groups or sessions (e.g., via cgroup or session ID) could close this gap.

- *ML-augmented classification.* Integrating lightweight ML models (e.g., decision trees trained on syscall sequences) could improve recall on ambiguous single-file cases without sacrificing the low false-positive rate achieved by the current rule-based approach.

Network Filtering. While per-IP rate limiting alone is insufficient against distributed or spoofed attacks, our system partially mitigates this through XDP-based connection tracking and request-response validation. However, this approach is protocol-specific and does not generalize to all traffic types. Fixed thresholds also introduce a tradeoff between false positives and missed low-rate attacks, and eBPF map size limits scalability under high flow volumes. Additionally, although XDP provides low-overhead filtering, more complex per-packet logic may introduce performance bottlenecks at extremely high packet rates [LSF+23].
Future work includes extending our validation beyond DNS to additional protocols, such as TCP and other application-layer protocols, as explored in prior work on protocol-aware and programmable network defenses that leverage per-protocol semantics for improved filtering effectiveness [VCP+20, ZLW+20]. We could also to scale

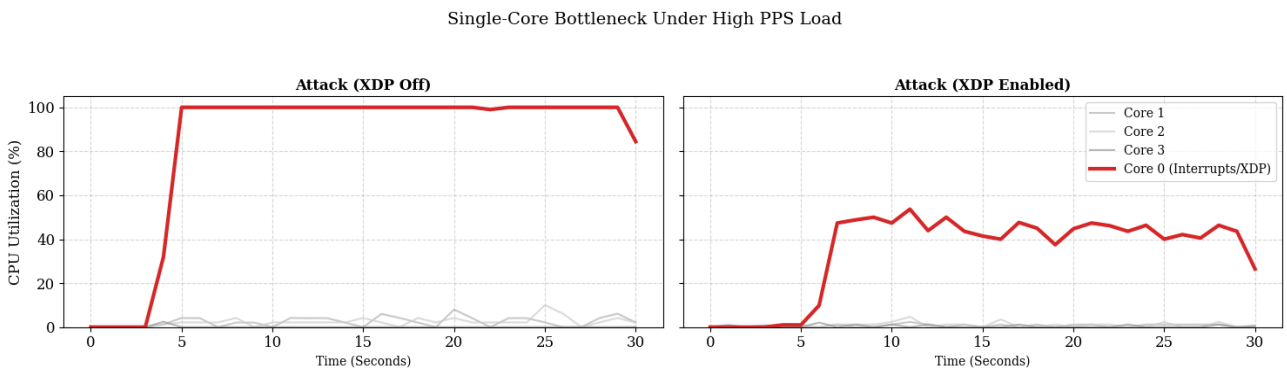


Figure 6: **CPU utilization under DNS reflection attack.** Without filtering (left), a single core becomes fully saturated while handling amplified traffic, indicating a bottleneck in packet processing. With XDP enabled (right), CPU usage is significantly reduced and more controlled, as malicious traffic is filtered early, preventing system-wide resource contention.

more complex setup with multiple devices that share information.

§7. Work Division

7.1. Ransomware

The code for this part can be found here.

- Anish contributed the kernel-space eBPF monitor, the heuristic design for ransomware behavior characterization, and the user-space agent including false-positive reduction techniques, the cumulative slow-burn profiling system, and the mitigation EDR response chain.
- Vaagish implemented the experimental harness and the testing suites to run end-to-end using the harness. He implemented the process-tree attribution of delegated writes. He performed experiments on the minimal whitelist entries needed to preserve the test performance and documented the justification for the entries in the broader whitelist. Lastly, he implemented part of the entropy gating of the heuristics.

7.2. Network Filtering

The code for this part can be found here.

- David conducted the experiments with the Raspberry Pi, researched and set up the XDP-based tagging for spoofing mitigation and the TCP SYN flood attacks with cookies.
- Danny researched and set up the XDP-Based Packet Filtering code and researched about the XDP-based tagging for spoofing mitigation.

References

[AA23] Amani Alraizza and Abdulmohsen Algarni. Ransomware detection using machine learning: A survey. *Big Data and Cognitive Computing*, 7(3):143, 2023. doi:10.3390/bdcc7030143.

[AAB+17] Manos Antonakakis, Tim April, Michael Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J Alex Halderman, Luca Invernizzi, Michalis Kallitsis, et al. Understanding the mirai botnet. In *26th USENIX security symposium (USENIX Security 17)*, pages 1093–1110, 2017.

[AS25] NF Abdulrahman and MJ Singh. Deep learning approaches for ddos attack detection in communication networks and iot: A comprehensive review. *Jurnal Kejuruteraan*, 37(1):323–333, 2025.

[BBCL19] Ossama Bouziani, Hafssa Benaboud, Achraf Samir Chamkar, and Saiida Lazaar. A comparative study of open source idss according to their ability to detect attacks. In *Proceedings of the 2nd International Conference on Networking, Information Systems & Security*, pages 1–5, 2019.

[CIS22] CISA and ACSC and CCCS and NCSC and FBI and NSA. 2021 top routinely exploited vulnerabilities. CISA Advisory AA22-117A, April 2022. Accessed: 2024. URL: https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-117a.

[CIS23a] CISA. Known exploited vulnerabilities catalog. Online, 2023. Accessed: 2024. URL: https://www.cisa.gov/known-exploited-vulnerabilities-catalog.

[CIS23b] CISA and FBI and MS-ISAC. Understanding ransomware threat actors: LockBit. CISA Advisory AA23-165A, June 2023. URL: https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-165a.

[Cis26] Cisco Systems. CVE-2026-20131: Cisco secure firewall management center software remote code execution vulnerability. Cisco Security Advisory cisco-sa-fmc-rce-NKhnULJh, March 2026. CVSS Score: 10.0; Added to CISA KEV Catalog March 19, 2026. URL: https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-fmc-rce-NKhnULJh.

[CTJS20] Cyril Cassagnes, Lucian Trestioreanu, Clement Joly, and Radu State. The rise of ebpf for non-intrusive performance monitoring. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, page 1–7. IEEE Press, 2020. doi:10.1109/NOMS47738.2020.9110434.

[Cyb21] Cybersecurity and Infrastructure Security Agency. Analysis report (AR21-126A): FiveHands ransomware. Technical report, CISA, May 2021. Accessed 2026-04-20. URL: https://www.cisa.gov/news-events/analysis-reports/ar21-126a.

[Cyb24] CyberVigilance. Kill, quarantine, remediate and rollback – SentinelOne. https://www.cybervigilance.uk/insights/sentinelone-kill-quarantine-remediate-and-rollback, 2024. Accessed: 2026-04-19.

[Ell14] Justin Ellingwood. How fail2ban works to protect services on linux server. *Digitalocean.com*, 7, 2014.

[FBD+11] M Ford, M Boucadair, A Durand, P Levis, and P Roberts. Issues with ip address sharing. Technical report, 2011.

[FMMR10] Alessandro Finamore, Marco Mellia, Michela Meo, and Dario Rossi. Kiss: Stochastic packet inspection classifier for udp traffic. *IEEE/ACM Transactions on Networking*, 18(5):1505–1515, 2010.

[FS98] Paul Ferguson and Daniel Senie. Network ingress filtering: Defeating denial of service attacks which employ ip source address spoofing. Technical report, 1998.

[HAC+24] Hassan Jalil Hadi, Mubashir Adnan, Yue Cao, Faisal Bashir Hussain, Naveed Ahmad, Mohammed Ali Alshara, and Yasir Javed. ikern: Advanced intrusion detection and prevention at the kernel level using ebpf. *Technologies*, 12(8):122, 2024.

[HHF26] David Hay, Mary Hogan, and Shir Landau Feibish. React: Reflection attack mitigation for asymmetric routing. *arXiv preprint arXiv:2601.06367*, 2026.

[HK23] Kosuke Higuchi and Ryotaro Kobayashi. Real-time defense system using ebpf for machine learning-based ransomware detection method. pages 213–219, 2023. URL: https://api.semanticscholar.org/CorpusID:267515126.

[IIBI24] Jahidul Ispahany, Md. Rafiqul Islam, M. Ali Babar, and Md. Zahidul Islam. Ransomware detection using machine learning: A review, research limitations and future directions. *IEEE Access*, 2024. doi:10.1109/ACCESS.2024.3397921.

[KMGG07] Georgios Kambourakis, Tassos Moschos, Dimitris Geneiatakis, and Stefanos Gritzalis. Detecting dns amplification attacks. In *International workshop on critical information infrastructures security*, pages 185–196. Springer, 2007.

[LSF+23] Jun Li, Devkishen Sisodia, Yebo Feng, Lumin Shi, Mingwei Zhang, Christopher Early, and Peter Reiher. Adaptive distributed filtering of ddos traffic on the internet. *arXiv preprint arXiv:2305.13296*, 2023.

[LXLG15] Chang Liu, Gang Xiong, Jie Liu, and Gaopeng Gou. Detect the reflection amplification attack based on udp protocol. In *2015 10th International Conference on Communications and Networking in China (ChinaCom)*, pages 260–265. IEEE, 2015.

[MJ93] Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *USENIX Winter 1993 Conference (USENIX Winter 1993 Conference)*, San Diego, CA, January 1993. USENIX Association. URL: https://www.usenix.org/conference/usenix-winter-1993-conference/bsd-packet-filter-new-architecture-user-level

[Mor21] Brenton Morris. RansomEXX, fixing corrupted ransom. https://profero.io/blog/ransomexx-fixing-corrupted-ransom, September 2021. Profero Incident Response, accessed 2026-04-20.

[NSJ16] Quamar Niyaz, Weiqing Sun, and Ahmad Y Javaid. A deep learning based ddos detection system in software-defined networking (sdn). *arXiv preprint arXiv:1611.07400*, 2016.

[PDF20] Petar Penkov, Eric Dumazet, and Stanislav Fomichev. Issuing syn cookies in xdp. In *Netdev 0x14*, 2020.

[PVMJ24] Tom Philippart, Bram Vrancken, Sam Michiels, and Wouter Joosen. Ransomware detection using machine learning in the Linux kernel, September 2024. URL:

https://arxiv.org/abs/2409.06452, arXiv:2409.06452.

[R+99] Martin Roesch et al. Snort: Lightweight intrusion detection for networks. In *Lisa*, volume 99, pages 229–238, 1999.

[Red26] Red Canary. T1486: Data Encrypted for Impact - Atomic Red Team. https://www.atomicredteam.io/docs/atomics/T1486, 2026. Accessed: 2026-03-28.

[RHM+20] Sivaramakrishnan Ramanathan, Anushah Hossain, Jelena Mirkovic, Minlan Yu, and Sadia Afroz. Quantifying the impact of blocklisting in the age of address reuse. In *Proceedings of the ACM Internet Measurement Conference*, pages 360–369, 2020.

[SK20] Fedor Sinitsyn and Vladimir Kuskov. RansomEXX trojan attacks linux systems. https://securelist.com/ransomexx-trojan-attacks-linux-systems/99279/, November 2020. Kaspersky Securelist, accessed 2026-04-20.

[SKK25] Arjun Sekar, Sameer G Kulkarni, and Joy Kuri. Learn: Leveraging ebpf and ai for ransomware nose out. In *2025 17th International Conference on COMmunication Systems and NETworks (COMSNETS)*, pages 1323–1328. IEEE, 2025.

[Ter23] Antonis Terefos. Cl0p ransomware targets linux systems with flawed encryption: Decryptor available. https://www.sentinelone.com/labs/cl0p-ransomware-targets-linux-systems-with-flawed-encryption-decryptor-available/, February 2023. SentinelOne SentinelLabs, accessed 2026-04-20.

[Tol25] Abdurrahman Tolay. ebpf-based real-time ddos mitigation for iot edge devices. *arXiv preprint arXiv:2508.00851*, 2025.

[VCP+20] Marcos AM Vieira, Matheus S Castanho, Racyus DG Pacífico, Elerson RS Santos, Eduardo PM Câmara Júnior, and Luiz FM Vieira. Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Computing Surveys (CSUR)*, 53(1):1–36, 2020.

[Wel04] Harald Welte. ct_sync: state replication of ip_conntrack. In *Linux Symposium*, page 537, 2004.

[YCR24] Sophia Yoo, Xiaoqi Chen, and Jennifer Rexford. {SmartCookie}: Blocking {Large-Scale}{SYN} floods with a {Split-Proxy} defense on programmable data planes. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 217–234, 2024.

[ZD23] Danyil Zhuravchak and Valerii Dudykevych. Real-time ransomware detection by using ebpf and natural language processing and machine learning. In *2023 IEEE 5th International Conference on Advanced Information and Communication Technologies (AICT)*, pages 1–4. IEEE, 2023.

[ZLW+20] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qianqian Li, Mingwei Xu, and Jianping Wu. Poseidon: Mitigating volumetric ddos attacks with programmable switches. In *the 27th Network and Distributed System Security Symposium (NDSS 2020)*, 2020.

[ZTP+23] Danylo Zhuravchak, Anastasiia Tolkachova, Andrian Piskozub, Valery Dudykevych, and Nataliia Korshun. Monitoring ransomware with Berkeley Packet Filter (BPF). In *Proceedings of the Cybersecurity Providing in Information and Telecommunication Systems (CPITS-2023-II)*, Kyiv, Ukraine, October 2023.

Supplementary Material

§A. Ransomware Experiment Detailed

Evaluation Philosophy. The ransomware monitor is evaluated using four complementary suites that probe different parts of the design. The legacy control suite checks whether narrow heuristic triggers still fire in isolation. The Atomic Red Team T1486 suite provides ATT&CK-aligned tool-level adversary emulations. The benign stress suite measures false positives under realistic legitimate workloads. The behavioral suite tests whether a non-whitelisted process that looks like ransomware in its file-system behavior is actually detected.

A.1. Legacy Control Suite

The legacy suite consists of self-authored control scenarios in `scenarios.csv`. These cases are intentionally small and targeted so that a miss usually maps to one heuristic or one integration boundary rather than to an ambiguous workload.

- `pos_touch_locked`: creates a file with a suspicious extension and tests the extension filter on file creation.
- `pos_rename_locked`: renames a benign file to `.locked` and tests suspicious rename handling.
- `pos_dd_urandom`: writes high-entropy bytes with `dd` and probes the entropy-plus-frequency path.
- `neg_touch_txt`, `neg_dd_zero`, `neg_plaintext_appends`, and `neg_archive_workload`: exercise benign file creation, zero-entropy writes, frequent low-entropy appends, and sustained archival I/O.

Interpretation. The legacy suite produced TP= 3, FP= 0, TN= 12, and FN= 6 across 21 runs. The most reliable positive control remains `pos_touch_locked`. The rename case is still weaker because short-lived rename activity can be missed if the event does not reach user space before the monitor is torn down. The `dd` case is now a systematic miss not because `dd` is trusted, but because a single process writing one opaque output file is outside the detector's intended ransomware boundary. This is a deliberate precision–recall tradeoff: the detector gives up sensitivity to a contrived one-file overwrite pattern in exchange for keeping the trust policy and behavioral heuristics aligned with multi-file ransomware behavior.

A.2. Atomic Red Team T1486 Suite

The Atomic suite uses the official Linux *Atomic Red Team* T1486 workloads and therefore gives us small, tool-level ATT&CK-style adversary emulations. These tests are valuable because they are externally sourced and technique aligned, but they are also intentionally atomic: each scenario exercises one tool on one file rather than a full multi-file ransomware kill chain.

- `pos_atomic_t1486_gpg_official`: encrypts a file with gpg.
- `pos_atomic_t1486_7z_official`: creates a password-protected archive with `7z`.
- `pos_atomic_t1486_ccrypt_official`: encrypts a file in place with `ccencrypt`.
- `pos_atomic_t1486_openssl_official`: encrypts a file with `openssl`.

Interpretation. The Atomic suite produced TP= 0 and FN= 12 across 12 runs. Under the current trust policy, that all-miss outcome is intentional rather than accidental: single invocations of trusted encryption and archival tools are no longer treated as a detection target. This makes the suite useful as a boundary marker instead of a score booster. The detector is not trying to infer malicious intent from one trusted tool touching one file; it is trying to recognize untrusted processes that behave like ransomware across many files and directories.

A.3. Benign Stress Suite

The benign stress suite is built from Phoronix-style workloads and is entirely negative-labeled. Its role is to measure false positives under realistic high-I/O tasks that legitimately produce compressed, encoded, or encrypted output and that therefore overlap with ransomware signals at the byte-pattern level.

- `neg_7z_compress`, `neg_zstd_compress`, `neg_gzip_compress`: benign compression workloads.
- `neg_gpg_encrypt_benign`, `neg_openssl_encrypt_benign`, `neg_ccencrypt_encrypt_benign`: benign single-file encryption workloads.
- `neg_ffmpeg_transcode`: benign media encoding.
- `neg_gcc_compile_burst`: benign compilation burst producing many object files.

Interpretation. The benign suite produced FP= 0 and TN= 24. This is the cleanest false-positive result the project has produced so far: all eight benign workload families stayed quiet across all three repeats. That quiet result is the mirror image of the Atomic suite. Once common encryption and archival tools are trusted as legitimate user utilities, the detector stops trying to distinguish benign from malicious one-file uses of those tools at the tool level. Instead, the burden of recall shifts to the behavioral suite, where the detector looks for ransomware-like kill chains from processes that are not trusted by name.

A.4. Behavioral Ransomware Simulation Suite

The behavioral suite is the most realistic evaluation of the detector's intended use case. Each scenario is a single process with a non-whitelisted kernel task name whose file-system behavior is inspired by documented Linux ransomware families, rather than merely invoking a known encryption utility once.

- `pos_ransim_walk`: a directory-walk encryptor inspired by the RansomEXX Linux variant, a single ELF binary that recursively encrypts files within target directories and appends encrypted key material needed for later decryption [SK20, Mor21].
- `pos_ransim_del`: an encrypt-then-delete workflow inspired by the broader encrypt-then-destroy pattern seen in ransomware operations, including FiveHands deleting recovery artifacts to inhibit restoration after encryption [Cyb21].
- `pos_ransim_rename`: an overwrite-and-rename workflow inspired by the Cl0p Linux variant, which encrypts targeted files in place via `mmap64/munmap` and uses family-specific output naming [Ter23].
- `pos_ransim_slow`: a throttled encryption variant used to probe sliding-window sensitivity.
- `pos_ransim_delegate`: a delegated-encryption workflow in which the parent orchestrator spawns a helper child encryptor.
- `pos_ransim_inplace`: a higher-fidelity in-place overwrite variant that destroys file contents without renaming outputs or adding suspicious extensions, more closely matching the syscall-level behavior described in published RansomEXX Linux analyses [Mor21].
- `pos_ransim_inplace_del`: an in-place overwrite-then-delete variant representing a more destructive wiper-ransomware hybrid pattern.
- `pos_ransim_inplace_slow`: a throttled in-place overwrite variant that removes cosmetic signals while probing the sliding-window robustness of the monitor.

We distinguish between *invariant* behavioral signals, which are actions the attacker must perform to achieve the attack objective, such as directory traversal and high-entropy overwrites of many files, and *cosmetic* signals that the attacker could trivially omit or vary, such as file-extension changes and ransom-note filenames. The higher-fidelity in-place variants (`inplace`, `inplace_del`, `inplace_slow`) deliberately omit these cosmetic signals, letting us test whether the detector catches the underlying behavior rather than superficial indicators.

Interpretation. The behavioral suite produced TP= 24 and FN= 0 across all 24 runs. This is the strongest evidence that the detector works well when an unknown process actually behaves like ransomware over multiple files and directories. The delegated case is particularly important because it validates the process-tree attribution extension: delegated child writes are attributed back to the active eligible non-whitelisted parent, so the parent accumulates the missing entropy evidence instead of the child silently absorbing it. The three higher-fidelity in-place variants strengthen the result further: they remain true positives even after removing cosmetic signals such as renamed outputs and suspicious extensions. Because the eBPF layer now emits ordinary open events as well as creations, the overwrite-oriented heuristics can observe these in-place passes directly rather than depending only on `O_CREAT`-seeded paths.

A.5. Cross-Suite Takeaways

Across the full tuned run, the combined ransomware evaluation produced TP= 27, FP= 0, FN= 18, and TN= 36, corresponding to precision 1.000, recall 0.600, specificity 1.000, and balanced accuracy 0.800. The detector now operates at a cleaner point in the precision–recall tradeoff: the false positives are gone entirely, but the cost is explicit in the Atomic and legacy misses.

The most informative cross-suite pattern is now the gap between tool-level and workflow-level evaluation. Under the current trust policy, the Atomic and benign suites separate cleanly: trusted one-file encryption and archival workloads are suppressed in both cases, so the Atomic suite becomes an all-miss tool-level boundary while the benign suite becomes an all-quiet false-positive check. The behavioral suite, in contrast, shows that the detector is much more decisive when it sees a process exhibit a fuller ransomware-style file-system kill chain.

The behavioral suite shows the intended recovery path. Rather than trying to classify every trusted or dual-use tool invocation, the detector performs best when it sees a non-whitelisted process traversing directories, touching many files, deleting originals, overwriting contents in place, or otherwise matching a ransomware kill chain. In the current tuned run it recovered all 24 of 24 behavioral positives, including the higher-fidelity in-place overwrite variants. The process-tree attribution extension improves that story further by letting a malicious parent inherit high-entropy write evidence from a delegated child helper without disabling the whitelist globally.

§B. Process Whitelist Design

The detector maintains a built-in whitelist of 32 process names whose legitimate operation overlaps strongly with ransomware-oriented heuristics. Whitelisted processes are skipped during analysis, subject to the integrity checks described in the main text (binary hash verification and process lineage validation). The entries fall into five functional categories.

Version Control. `git` is whitelisted because cloning, checkout, reset, clean, and garbage collection legitimately materialize, rewrite, and delete many files and packed objects in a single workflow, resembling destructive bulk file activity.

Package Managers and Dependency Installers. Twelve entries cover system and language-level package managers: `apt`, `apt-get`, `dpkg`, `dnf`, `yum`, `pacman`, `snap`, `flatpak`, `pip`, `pip3`, `npm`, and `yarn`. All of these perform broad file replacement and cleanup across many paths during install and upgrade operations, which overlaps with the detector's delete-heavy and multi-file write heuristics.

Compression and Encryption. Twelve entries cover tools that produce high-entropy output or delete originals: `gzip`, `bzip2`, `xz`, `zstd`, `lz4`, `lzop`, `pigz`, `pbzip2`, `pixz` (compression), and `gpg`, `ccencrypt`, `openssl` (encryption). These tools emit opaque, ciphertext-like outputs that directly overlap with the entropy-based write heuristics, and several optionally delete the source file after producing compressed or encrypted output.

Synchronization and Overwrite-Capable Tools. Four entries cover tools with bulk write-and-delete semantics: `rsync`, `rclone`, `mogrify`, and `logrotate`. These combine broad file writes with legitimate bulk deletion, rename, or in-place overwrite behavior.

Databases and Datastores. Three entries cover database engines: `postgres`, `mysqld`, and `mongod`. These maintain redo logs, tablespaces, journals, and temporary files in patterns that overlap with repeated opaque multi-file writes.

Design Rationale. The whitelist explicitly does *not* trust generic editors, compiler stages, schedulers, media encoders, or simple one-output tools. The remaining 32 entries are those that still pose a realistic false-positive threat under the detector's surviving ransomware-oriented heuristics. The trust policy favors suppressing false positives from single-file invocations of common encryption and archival tools over attempting to classify those invocations as ransomware on behavior alone.

§C. Minimal Whitelist Ablation

While the built-in whitelist contains 32 entries for broad deployment, the measured test context admits a much smaller policy. We performed a systematic ablation to find the minimal whitelist that preserves the experimental operating point.

Goal. The target was a whitelist that is (1) small enough to explain entry by entry, (2) strong enough to keep the benign suite quiet, and (3) narrow enough that removing any single remaining entry measurably changes the results.

Minimal Policy. The final reduced whitelist was tested through a config-level trust policy (not by changing the detector's built-in defaults) and kept exactly five entries: `gpg`, `gzip`, `zstd`, `ccencrypt`, and `openssl`. The same baseline used the detector's process-tree attribution policy, allowing delegated child writes to be attributed to an eligible suspicious parent.

Baseline Result. With the five-entry policy, the tuned 3-repeat full run produced the results shown in Table 4.

Suite	TP	FP	TN	FN
Legacy	6	0	12	3
Atomic T1486	0	0	0	12
Benign	0	0	24	0
Behavioral	24	0	0	0
Overall	**30**	**0**	**36**	**15**

Table 4: Baseline results with the minimal five-entry whitelist policy.

Ablation Results. We ran a full ablation over the five-entry policy: the baseline, then baseline-minus-one for each entry, across all four suites with three repeats. Table 5 summarizes the results.

Interpretation. The ablation pattern was unusually clean. Every one of the five entries was indispensable: removing any single entry caused exactly one benign workload family to flip from TN= 3/3 to FP= 3/3, while no legacy, Atomic, or behavioral count changed under any removal. The per-entry flips were one-to-one:

- Removing `gpg` broke only `neg_gpg_encrypt_benign`.
- Removing `gzip` broke only `neg_gzip_compress`.
- Removing `zstd` broke only `neg_zstd_compress`.
- Removing `ccencrypt` broke only `neg_ccencrypt_encrypt_benign`.
- Removing `openssl` broke only `neg_openssl_encrypt_benign`.

This confirms that, in the measured test context, the five-entry set is the *minimal* whitelist that preserves the final operating point. Each entry suppresses exactly one benign workload family, and no entry is redundant. The larger 32-entry built-in whitelist provides additional coverage for deployment scenarios beyond the test suite, but the ablation demonstrates that the experimental results depend on exactly these five trust decisions.

Variant	Legacy TP/FP/TN/FN	T1486 TP/FP/TN/FN	Benign TP/FP/TN/FN	Behavioral TP/FP/TN/FN
Baseline	6 / 0 / 12 / 3	0 / 0 / 0 / 12	0 / 0 / 24 / 0	24 / 0 / 0 / 0
Remove `gpg`	6 / 0 / 12 / 3	0 / 0 / 0 / 12	0 / 3 / 21 / 0	24 / 0 / 0 / 0
Remove `gzip`	6 / 0 / 12 / 3	0 / 0 / 0 / 12	0 / 3 / 21 / 0	24 / 0 / 0 / 0
Remove `zstd`	6 / 0 / 12 / 3	0 / 0 / 0 / 12	0 / 3 / 21 / 0	24 / 0 / 0 / 0
Remove `ccencrypt`	6 / 0 / 12 / 3	0 / 0 / 0 / 12	0 / 3 / 21 / 0	24 / 0 / 0 / 0
Remove `openssl`	6 / 0 / 12 / 3	0 / 0 / 0 / 12	0 / 3 / 21 / 0	24 / 0 / 0 / 0

Table 5: Full whitelist ablation: baseline vs. baseline-minus-one for each of the five entries, across all four suites (3 repeats each).